

Rozwiązywanie równania nieliniowego metodą Newtona w arytmetyce zmiennopozycyjnej oraz przedziałowej

1. Zastosowanie

Procedury rozwiązują równanie nieliniowe postaci $f(x) = 0$ metodą iteracyjną Newtona z wykorzystaniem klasycznej arytmetyki zmiennopozycyjnej wysokiej precyzji oraz w zmiennopozycyjnej arytmetyce przedziałowej.

2. Opis metody

Metoda Newtona (metoda stycznych) jest iteracyjną metodą wyznaczania przybliżonego rozwiązania równania:

$$f(x) = 0$$

Dla klasycznej arytmetyki zmiennopozycyjnej proces iteracyjny opisuje wzór:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}, \quad i = 0, 1, 2, \dots$$

gdzie:

x_0 – przybliżenie początkowe,

$f(x)$ – funkcja,

$f'(x)$ – pochodna funkcji.

Proces iteracyjny kończy się, gdy spełniony jest warunek:

$$\frac{|x_{i+1} - x_i|}{\max(|x_{i+1}|, |x_i|)} < \epsilon$$

lub gdy:

$$x_{i+1} = x_i = 0$$

gdzie ϵ oznacza zadaną dokładność obliczeń.

W przypadku **arytmetyki przedziałowej**, w każdej iteracji dla bieżącego przedziału X_i wyznacza się jego środek: $m = \text{mid}(X_i)$. Następnie obliczany jest przedziałowy operator Newtona zdefiniowany wzorem:

$$N(X_i) = m - \frac{f(m)}{f'(X_i)}$$

Nowym przybliżeniem przedziałowym staje się część wspólna (przecięcie) przedziału poprzedniego oraz wyniku działania operatora:

$$X_{i+1} = X_i \cap N(X_i)$$

Proces przerywa się, gdy szerokość przedziału X_{i+1} jest mniejsza od ϵ lub gdy przecięcie przedziałów jest zbiorem pustym. (Dla trybu danych rzeczywistych, proces przerywa się, gdy różnica między kolejnymi przybliżeniami m jest mniejsza od ϵ , bez wyznaczania części wspólnej).

3. Wywołanie procedur

```
NewtonZwykly(f, df, wejscie, max_iter, epsilon);  
NewtonPrzedzialowy(f_int, df_int, wejscie, max_iter, tryb, epsilon);
```

4. Dane

`f` – funkcja języka C++, która oblicza wartość funkcji $f(x)$,
`df` – funkcja języka C++, która oblicza wartość pochodnej $f'(x)$,
`f_int` – funkcja przedziałowa obliczająca $f(X)$ w arytmetyce przedziałowej,
`df_int` – funkcja przedziałowa obliczająca $f'(X)$ w arytmetyce przedziałowej,
`wejscie` – ciąg znaków określający początkowe przybliżenie x_0 (np. 1.5) lub końce przedziału początkowego oddzielone średnikiem (np. 1.0;3.0),
`max_iter` – maksymalna dozwolona liczba iteracji w metodzie Newtona,
`epsilon` – błąd względny rozwiązania (zadana dokładność).

5. Wyniki

Kolejne wyniki iteracji są przekazywane na standardowe wyjście programu (`cout`):

- ciąg przybliżonych punktów x będących rozwiązaniem równania (dla arytmetyki zwykłej),
- ciąg przedziałów (określających gwarantowane granice rozwiązania) wraz z ich szerokością (dla arytmetyki przedziałowej).

6. Inne parametry

`tryb` – zmienna całkowita, która określa rodzaj wywoływanej arytmetyki w obrębie procedury głównej: 1 (zwykła), 2 (przedziałowa dla danych rzeczywistych) lub 3 (przedziałowa dla danych przedziałowych).

7. Typy parametrów

```
string: wejscie  
int: max_iter, tryb  
double: epsilon  
F_REAL: f  
DF_REAL: df  
F_INT: f_int  
DF_INT: df_int
```

8. Identyfikatory nielokalne

`mpreal` – nazwa typu zmiennopozycyjnego o wysokiej precyzji zdefiniowanego w bibliotece MPFR,

Interval<double> – nazwa typu generycznego reprezentującego przedział, zdefiniowanego w bibliotece Interval.h,

F_REAL – identyfikator typu proceduralnego zdefiniowany następująco: typedef double (*F_REAL)(double);

DF_REAL – identyfikator typu proceduralnego zdefiniowany następująco: typedef double (*DF_REAL)(double);

F_INT – identyfikator typu proceduralnego zdefiniowany następująco: typedef Interval<double> (*F_INT)(Interval<double>);

DF_INT – identyfikator typu proceduralnego zdefiniowany następująco: typedef Interval<double> (*DF_INT)(Interval<double>);

9. Tekst procedur

```
1 void NewtonZwykly(F_REAL f, DF_REAL df, string wejscie, int max_iter,
2 double epsilon) {
3     mpreal::set_default_prec(256);
4     mpreal x = wejscie;
5
6     for (int i = 0; i < max_iter; i++) {
7         double current_x = x.toDouble();
8         double fx = f(current_x);
9         double dfx = df(current_x);
10
11         if (abs(dfx) < 1e-20) {
12             cout << "Blad: Pochodna bliska zeru." << endl;
13             break;
14         }
15
16         mpreal x_nowy = x - fx / dfx;
17         cout << "Iteracja " << i + 1 << " -> x = " << scientific <<
18             setprecision(17) << x_nowy << endl;
19
20         if (abs(x_nowy - x) < epsilon) {
21             cout << "SUKCES: Osiagnieto zadana dokladnosc (epsilon = "
22                 << epsilon << ")." << endl;
23             break;
24         }
25
26         x = x_nowy;
27         if (i == max_iter - 1) cout << "Osiagnieto limit iteracji." <<
28             endl;
29     }
30 }
31
32 void NewtonPrzedzialowy(F_INT f_int, DF_INT df_int, string wejscie, int
33 max_iter, int tryb, double epsilon) {
34     Interval<double>::Initialize();
35     Interval<double> X;
36
37     if (tryb == 2) {
38         X.a = LeftRead<double>(wejscie);
39         X.b = RightRead<double>(wejscie);
40     } else {
41         size_t sep = wejscie.find(';');
42         if (sep != string::npos) {
```

```

38         string L_str = wejscie.substr(0, sep);
39         string R_str = wejscie.substr(sep + 1);
40         X.a = LeftRead<double>(L_str);
41         X.b = RightRead<double>(R_str);
42     } else {
43         X.a = LeftRead<double>(wejscie);
44         X.b = RightRead<double>(wejscie);
45     }
46 }
47
48 for (int i = 0; i < max_iter; i++) {
49     double m = X.Mid();
50
51     Interval<double> m_interval;
52     m_interval.a = m;
53     m_interval.b = m;
54     Interval<double> fm = f_int(m_interval);
55
56     Interval<double> dfX = df_int(X);
57
58     if (dfX.a <= 0.0 && dfX.b >= 0.0) {
59         cout << "Bład: Zero znajduje sie w przedziale pochodnej (
60             dzielenie przez zero)." << endl;
61         break;
62     }
63
64     Interval<double> N = m_interval - (fm / dfX);
65
66     Interval<double> X_nowy;
67
68     if (tryb == 3) {
69         X_nowy.a = max(X.a, N.a);
70         X_nowy.b = min(X.b, N.b);
71
72         if (X_nowy.a > X_nowy.b) {
73             cout << "Bład: Przeciecie przedzialow jest puste. Brak
74                 pierwiastka w podanym obszarze." << endl;
75             break;
76         }
77     } else {
78         X_nowy = N;
79     }
80
81     X = X_nowy;
82     string lewy_koniec, prawy_koniec;
83     X.IEndsToStrings(lewy_koniec, prawy_koniec);
84     double szerokosc = X.b - X.a;
85
86     cout << "Iteracja " << i + 1 << " -> przedzial: ["
87         << lewy_koniec << " ; " << prawy_koniec
88         << "]" (szerokosc: " << scientific << setprecision(17) <<
89         szerokosc << ")" << endl;
90
91     if (tryb == 3) {
92         if (szerokosc < epsilon) {
93             cout << "SUKCES: Osiagnieto zadana dokladnosc (szerokosc
94                 przedzialu < epsilon)." << endl;
95             break;
96         }
97     }
98 }

```

```

92     }
93     } else {
94         if (abs(X.Mid() - m) < epsilon) {
95             cout << "SUKCES: Osiagnieto zadana dokladnosc (roznica
96                 przyblizen < epsilon)." << endl;
97             break;
98         }
99     }
100     if (i == max_iter - 1) cout << "Osiagnieto limit iteracji." <<
        endl;
101 }
102 }

```

10. Przykłady

a) Równanie nieliniowe – Tryb 1: Klasyczna arytmetyka (zwykła):

Równanie: $x^2 - 4 = 0$

Dane: tryb = 1, wejscie = 1.0, max_iter = 100, epsilon = 1e-10

Wyniki:

Iteracja 1 -> x = 2.5000000000000000e+00

Iteracja 2 -> x = 2.0499999999999999e+00

Iteracja 3 -> x = 2.0006097560975610e+00

Iteracja 4 -> x = 2.0000000929222949e+00

Iteracja 5 -> x = 2.0000000000000002e+00

Iteracja 6 -> x = 2.0000000000000000e+00

SUKCES: Osiagnieto zadana dokladnosc (epsilon = 1.0000000000000000e-10).

b) Równanie nieliniowe – Tryb 2: Arytmetyka przedziałowa (dane rzeczywiste):

Równanie: $x^2 - 4 = 0$

Dane: tryb = 2, wejscie = 1.0, max_iter = 100, epsilon = 1e-10

Wyniki:

Iteracja 1 -> przedzial: [2.5000000000000000E0 ; 2.5000000000000000E0]
(szerokosc: 0.0000000000000000e+00)

Iteracja 2 -> przedzial: [2.0499999999999998E0 ; 2.0500000000000003E0]
(szerokosc: 4.4408920985006261e-16)

Iteracja 3 -> przedzial: [2.0006097560975608E0 ; 2.0006097560975614E0]
(szerokosc: 4.4408920985006261e-16)

Iteracja 4 -> przedzial: [2.0000000929222943E0 ; 2.0000000929222952E0]
(szerokosc: 8.8817841970012523e-16)

Iteracja 5 -> przedzial: [2.00000000000000017E0 ; 2.00000000000000027E0]
(szerokosc: 8.8817841970012523e-16)

Iteracja 6 -> przedzial: [1.9999999999999997E0 ; 2.0000000000000005E0]
(szerokosc: 6.6613381477509392e-16)

SUKCES: Osiagnieto zadana dokladnosc (roznica przyblizen < epsilon).

c) Równanie nieliniowe – Tryb 3: Arytmetyka przedziałowa (dane przedziałowe):

Równanie: $x^2 - 4 = 0$

Dane: tryb = 3, wejscie = 1.2;3.7, max_iter = 100, epsilon = 1e-10

Wyniki:

Iteracja 1 -> przedzial: [1.6156249999999998E0 ; 2.1793918918918922E0]
(szerokosc: 5.63766891891892330e-01)

Iteracja 2 -> przedzial: [1.9891536656629549E0 ; 2.0211329558302947E0]
(szerokosc: 3.19792901673396912e-02)

Iteracja 3 -> przedzial: [1.9999653054000467E0 ; 2.0000472341692502E0]
(szerokosc: 8.19287692033210391e-05)

Iteracja 4 -> przedzial: [1.9999999998814064E0 ; 2.0000000001382437E0]
(szerokosc: 2.56837218159944314e-10)

Iteracja 5 -> przedzial: [1.9999999999999995E0 ; 2.0000000000000000E0]
(szerokosc: 4.44089209850062616e-16)

SUKCES: Osiagnieto zadana dokladnosc (szerokosc przedzialu < epsilon).

d) Równanie nieliniowe – Sytuacja wyjątkowa (brak pierwiastka w przedziale):

Równanie: $x^2 - 4 = 0$

Dane: tryb = 3, wejscie = 4.0;5.0, max_iter = 100, epsilon = 1e-10

Wyniki:

Blad: Przeciecie przedzialow jest puste. Brak pierwiastka w podanym obszarze.